TITAN-ASCII
Communication Manual

Revision 6.0L

First Edition, Jan 2019

Table of Contents

1. INTRODUCTION 4
2. COMMUNICATION OVERVIEW 5
   2.1. RS-485 & USB COMMUNICATION PARAMETERS 5
   2.2. ETHERNET COMMUNICATION PARAMETERS 5
   2.3. COMMUNICATION PROTOCOL 6
3. TITAN-ASCII PROTOCOL 7
   3.1. COMMUNICATION MODE 0 7
   3.2. COMMUNICATION MODE 1 9
   3.3. COMMUNICATION MODE 2 10
   3.4. COMMUNICATION MODE 3 12
APPENDIX A. TITAN-ASCII COMMAND 13
   A.1. STATUS COMMANDS 14
   A.2. LED COMMANDS 15
   A.3. MOTION COMMANDS 16
   A.4. GAIN AND FILTER COMMANDS 18
   A.5. STANDALONE PROGRAM COMMANDS 19
   A.6. LIMIT COMMANDS 20
   A.7. IO COMMANDS 21
   A.8. VARIABLE COMMANDS 22
   A.9. FAULT MONITORING COMMANDS 23
   A.10. MISCELLANEOUS COMMANDS 25
   A.11. PROBE COMMANDS 26
   A.12 USING TITAN-ASCII COMMAND 27
      A.12.1. Position, Velocity, Position Error 27
      A.12.2. In-Pos 28
      A.12.3. Velocity 28
      A.12.4. Motor Current 28
      A.12.5. Motor Status 29
      A.12.6. Motor Fault 30
      A.12.7. Enable Servo / Open Loop Position Hold 32
      A.12.8. Jogging 33
      A.12.9. Homing 34
      A.12.10. Target Move 35
      A.12.11. Gains 36
      A.12.12. Digital IO 37
      A.12.13. LED 37
      A.12.14. Variable 38
      A.12.15. Program 39
      A.12.16. Miscellaneous Commands 40
      A.12.17. Multi-axis Synchronous Start Move 41
      A.12.18. Motion Probe 42

1. Introduction

The TITAN is a universal motor driver-controller that supports most of the commonly used motors in the automation industry including:

- 2 Phase Stepper Motor.
- 3 Phase Brushless Rotary Servo Motor.
- 3 Phase Brushless Linear Servo Motor.
- DC Voice Coil Motor.

The TITAN Servo technology has been packaged into various products as listed below with different communication options:

- TITAN-SVX-ETH – USB/RS485/ETHERNET.
- TITAN-SVX-SC2 – USB/RS485/CAN.
- TITAN-SVX-ET2 – USB/DUAL PORT ETHERNET.
- TITAN-SVX-5.0 – USB/WIFI/BT.
- TITAN-NXS-SCX – USB/RS485/CAN.
- TITAN-NXS-SPX_ USB.
- TITAN-IMX-23 – RS485.
- TITAN-IMX-34 – RS485.
- TITAN-CORE with baseboard products – USB/RS485.

This manual outlines the details of the TITAN-ASCII Communication Protocol, an ASCII text-based common communication protocol implemented in all the TITAN Servo Products.

2. Communication Overview

2.1. RS-485 & USB Communication Parameters

Communication with TITAN can be done using an RS-485 multi-drop network or a direct USB connection.

For USB Communication, TITAN uses a Virtual COM Port converter chip and will display as a serial communication port on the PC. Before using a USB connection, confirm that the USB Virtual COM Port driver is installed properly on the PC.

The following are the communication settings for both RS-485 and USB connections:

Parameter	Setting
Baud Rate	115,200
Byte Size	8 bits
Parity	None
Flow Control	None
Stop Bit	1

Note that the above settings are fixed and are used for all the supported protocols including MODBUS-ASCII and MODBUS-RTU.

2.2. Ethernet Communication Parameters

Communication with TITAN can also be done using an Ethernet connection if the TITAN controller has an Ethernet communication port.

Ethernet communication with TITAN is done using standard socket communication.

Before attempting an Ethernet communication, be sure to setup the IP address of the TITAN.

The Ethernet port that is used on the TITAN is 5000.

2.3. Communication Protocol

TITAN supports the following protocols:

Communication Mode	Protocol
0	TITAN-ASCII (no CRC)
1	TITAN-ASCII (no CRC) with No Error Reply
2	TITAN-ASCII (with CRC)
3	TITAN-ASCII (with CRC) with No Error Reply
4	MODBUS-ASCII (with LRC)
5	MODBUS-RTU (with CRC)

- MODBUS protocols are only available on RS-485 and USB communication.
- For details of the MODBUS Protocol, please refer to the TITAN MODBUS Communication Protocol Manual.
- Default protocol is TITAN-ASCII (no CRC).
- TITAN Windows UI software supports the Communication Mode 0 or 1.

3. TITAN-ASCII Protocol

3.1. Communication Mode 0

Communication Mode 0 is the TITAN-ASCII Protocol with Error Reply.

Command from Master to TITAN

	Start	NetID	Sep	Command	End1	End2
Char #	1	2	1	XX…Xn	1	1
ASCII Char	@	NN	**:**	[Command String with "**;**" delimiters]	CR	LF

Reply from TITAN to Master

	Start	NetID	Sep	Reply	End1	End2
Char #	1	2	1	XX	1	1
ASCII Char	#	NN	**:**	[Reply String with "**;**" delimiters]	CR	LF

Note: ASCII value of CR = 13, ASCII value of LF = 10.

- The command will be processed by the receiving TITAN only if the Start Character, Network ID, and Separation Characters are valid. If the characters do not match, TITAN will ignore the command, and no reply will be sent from the TITAN unit.
- The Network ID range is from 01 to 99.
- Any invalid command following valid start characters will result in a reply message. If there is any error in processing the command, an error reply will be sent.
- After sending a command string with @ as start and LF as end characters, the master must release the RS485 signal after the LF character and must not send any characters until a complete reply is sent from TITAN. If a string or character is sent immediately after the command string, the reply string will collide with the message sent from the master and will result in a garbled string.
- If the RS485 line cannot be released immediately by the master after sending the command, use the **TXDELAY** command to set the delay time for TITAN to delay before the reply is sent back to the master.
- The maximum length of the command and the reply is 256 characters.
- Multiple commands can be sent in one string. Commands must be separated by a semicolon character ("**;**").

Example 1:

A query command **EX** for the encoder position of the motor.

Command from Master to TITAN:

@01:EX[CR][LF]

Reply from TITAN to Master

#01:EX=12345[CR][LF]

Example 2:

A query command for the encoder position and the velocity of the motor in one command line. Note that the two commands are sent and are separated by a delimiter character (“**;**”)

Command from Master to TITAN

@01:EX;VX[CR][LF]

Reply from TITAN to Master

#01:EX=12345;VX=0[CR][LF]

Example 3:

Unknown command ZZZ is sent. A reply with an error message is sent back from TITAN.

Command from Master to TITAN

@01:ZZZ[CR][LF]

Reply from TITAN to Master

#01:COMERR2[CR][LF]

3.2. Communication Mode 1

Communication Mode 1 is the TITAN-ASCII protocol with no Error Reply.

Communication Mode 1 is exactly the same as Communication Mode 0 except that only the valid replies are sent back from TITAN.

Communication Mode 1 can be useful when the RS485 network has devices other than TITAN and helps in reducing the network traffic.

Example 1:

An unknown command TEST is sent. TITAN does not reply with any error message.

Command from Master to TITAN
@01:TEST[CR][LF]

Reply from TITAN to Master
[No reply will be sent from the TITAN]

3.3. Communication Mode 2

Communication Mode 2 is the TITAN-ASCII protocol with CRC Checksum with error reply.

Command from Master to TITAN

	Start	NetID	Sep	Command	Sep	CRC	End1	End2
Char #	1	2	1	XX	1	4	1	1
ASCII Char	@	nn	**:**	[Command String with "**;**" delimiters]	*****	[CCCC]	CR	LF

Reply from TITAN to Master

	Start	NetID	Sep	Reply	Sep	CRC	End1	End2
Char #	1	2	1	XX	1	4	1	1
ASCII Char	#	nn	**:**	[Reply String with "**;**" delimiters]	*****	[CCCC]	CR	LF

Note: ASCII value of CR = 13, ASCII value of LF = 10.

- The command will be processed only if the Start Character, Network ID, and Separation Characters are valid. If the characters do not match, no reply will be sent from TITAN.
- The Network ID range is from 01 to 99.
- Any invalid command following valid start characters will result in a reply with a command error message.
- CRC Checksum characters are four ASCII characters that represent CRC-16 Checksum values of the characters including the Start, Network ID, Separation character, and Command or Reply characters.
- The "*" character separates the command or the reply message from the CRC Checksum characters.
- CRC-16 uses the following parameters in the calculation of CRC Checksum value:
    - Input Type: ASCII.
    - Polynomial: 0x8005.
    - Initial Value: 0xFFFF.
    - Final XOR: 0x0000.

Example 1:

A query command for the encoder position of the motor with four CRC characters appended to the command.

The reply from the TITAN also comes with the with CRC Checksum characters.

Command from Master to TITAN

@01:EX*4453[CR][LF]

Reply from TITAN to Master

#01:EX=830141*868D[CR][LF]

Example 2:

Incorrect CRC value characters are sent, and an error message is sent back from the TITAN.

Command from Master to TITAN

@01:EX*1234[CR][LF]

Reply from TITAN to Master

#01:COMERR1*F960[CR][LF]

Example 3:

Multiple commands are sent on a single string along with the CRC Checksum characters. The reply comes back with the CRC characters.

Command from Master to TITAN

@01:EX;VX*868C[CR][LF]

Reply from TITAN to Master

#01:EX=830141;VX=0*4D60[CR][LF]

3.4. Communication Mode 3

Communication Mode 3 is exactly the same as Communication Mode 2, with the exception that only the valid commands are replied.

For example, sending the wrong CRC value characters will not result in a reply.

Wrong commands will also not result in a reply.

Example 1:

Command from Master to TITAN

@01:EX*1234[CR][LF]

No reply from TITAN will be sent since the CRC check characters are invalid.

NOTE:
Only the valid commands will be replied in Communication Mode 3.

APPENDIX A. TITAN-ASCII Command

Notes:

1. All TITAN-ASCII commands (Communication Mode 0, 1, 2, and 3) must start with the start character @, two Network ID characters, and a semicolon character and must end with CR and LF characters.

2. All valid commands are replied to with the start character #, two Network ID characters, and a semicolon character with ending CR and LF characters.

3. Invalid commands are replied to with an error message in Communication Mode 0 and 2 only.

4. After the command is sent, the master must release the RS-485 line immediately and wait for the complete reply message before replying in order to avoid any collision in the message. Sending characters or messages while the reply is being sent or an RS-485 line not being released when the reply message is sent will result in an unrecognizable and garbled message.

5. Commands can be Read-Only, Write-Only, or Read/Write, denoted by R, W, RW.

6. All valid commands are replied to with a message.

7. Invalid commands may or may not be replied to depending on the communication mode.

NOTE:
Besides the published TITAN-ASCII commands listed in this document, additional non-published commands exist. Users of TITAN are highly recommended to use only the published commands. Using the wrong commands may create an unexpected situation which may result in damage to the products and a potentially dangerous situation for the user.

A.1. Status Commands

Command	RW*	Description	Example
CURQA	R	Motor current Q	@01:CURQA #01:CURQA=0.03975
CURDA	R	Motor current D	@01:CURDA #01:CURDA=0
EET	RW	Encoder Error Type Bit 0 – Encoder Count Per Index Error Bit 1 – Align Phase Error Bit 2 – Homing Index No Index Bit 3 – Encoder Break Bit 4 – No Encoder for DC Bit 5 – No Encoder for PMSM Bit 6 – No Encoder for Step	@01:EET #01:EET=1
EIPOS	R	Encoder Count Per Index Trigger	@01:EIPOS #01:EIPOS=4000
EX	RW	Encoder position	@01:EX #01:EX=0
FLT	R	Fault condition. Bit 0 – Negative Limit Error Bit 1 – Positive Limit Error Bit 2 – Position Error Bit 3 – Current Error Bit 4 – Hall Sensor Error Bit 5 – Over Current Error Bit 6 – Over Voltage Error Bit 7 – Under Voltage Error Bit 8 – Overheat Error Bit 9 – Reserved Bit 10 – Emergency Input (HW Input) Bit 11 – Encoder Error (HW Input) Bit 12 – Reserved Bit 13 – Reserved Bit 14 – Encoder Drift / Bias Angle Error Bit 15 – Flash Memory Error **Return value in Hex 0x0 format**	@01:FLT #01:FLT=0x0
INPOS	RW	In-position value.	@01:INPOS #01:INPOS=200
MST	R	Returns current motor status. Bit 0 – Enabled Bit 1 – In Position Bit 2 – Moving Bit 3 – In Fault Bit 4 – Homed **Return value in Hex 0x0 format**	@01:MST #01:MST=0x0
PERR	R	Position error	@01:PERR #01:PERR=0
POSD	R	Target position	@01:POSD #01:POSD=0
VPROF	R	Target velocity	@01:VPROF #01:VPROF=0
VX	R	Actual velocity (estimated from encoder)	@01:VX #01:VX=0

A.2. LED Commands

Command	RW*	Description	Example
LED	RW	Blue LED on the face of TITAN	@01:LED #01:LED=1
RGB	RW	Internal RGB LED Bit 0 – Red LED Bit 1 – Green LED Bit 2 – Blue LED	@01:RGB #01:RGB=4

A.3. Motion Commands

Command	RW*	Description	Example
ACC	RW	Target acceleration	@01:ACC #01:ACC=100
CM	W	Synchronous multi-axis motion start command. Use only with address 0 for broadcast. Each target position is represented by 8 hex value characters from address 1 to 6 axis total.	@00:CM=00000BB8 00001138 00000000 00000000 00000000 00000000
DEC	RW	Target deceleration	@01:DEC #01:DEC=100
ECLEARX	W*	Clear Fault	@01:ECLEARX #01:ECLEARX=1
HMODE	RW	Homing Mode 0 – plus home 1 – minus home 2 – plus limit 3 – minus limit 4 – plus home + encoder index 5 – minus home + encoder index 6 – plus limit + encoder index 7 – minus limit + encoder index 8 – plus home using index channel 9 – minus home using index channel	@01:HMODE #01:HMODE=0
HOMEX	W*	Perform Home. See motion return code.	@01:HOMEX #01:HOMEX=1
HSPD	RW	Target velocity	@01:HSPD #01:HSPD=10
JERK	RW	The rate of change in acceleration while moving in a parabolic motion profile. Units in rpm/sec^2 or mm/sec^3	@01:JERK #01:JERK=265462
JOGV	W*	Jog to target velocity. See motion return code.	@01:JOGV=2000 #01:JOGXV=1
JOGXN	W*	Jog in Negative Direction. See motion return code.	@01:JOGXN #01:JOGXN=1
JOGXP	W*	Jog in Positive Direction. See motion return code.	@01:JOGXP #01:JOGXP=1
NOPROF	RW	Enable Profile or No profile motion. When no profile is selected, acceleration and deceleration and constant velocity will not be used.	@01:NOPROF=0 #01:NOPROF=0
OLPHOLD	RW	Open Loop Position Hold (Valid for Stepper Only) Value 0 – Full time closed loop Value 1-100 – Percentage current hold at Enabled and Not Moving.	@01:OLPHOLD #01:OLPHOLD=0
PFT	RW	Profile Type TITAN can operate in 3 different motion profiles: 1 – trapezoidal 2 – s-curve 3 – parabolic	@01:PFT #01:PFT=2
STOPX	W*	Stop Motion	@01:STOPX

			#01:STOPX=1
SVOFF	W*	Disable Motor or Servo Off	@01:SVOFF #01:SVOFF=1
SVON	W*	Enable Motor or Servo On	@01:SVON #01:SVON=1
SVRST	W*	Servo reset	@01:SVRST #01:SVRST=1
X	W	Perform Target position move. See motion return code.	@01:X=1000 #01:X=1

Following are return codes for the motion commands X, JOGV, JOGN, JOGP, and HOMEX.

Motion Return Codes

Return Code	Description
1	Motion Successful
0	Motion in progress.
4	Plus Limit Error
5	Minus Limit Error
6	Already at target position
11	Motor is disabled
12	Motor is in Limit Error
23	Motion is Blocked State

A.4. Gain and Filter Commands

Command	RW*	Description	
CGAINF	RW	Current Control Firmness. Range 0 to 100	@01:CGAINF #01:CGAINF=80
IGAINF	RW	Integral Control Firmness. Range 0 to 100	@01:IGAINF #01:IGAINF=80
PGAINF	RW	Position Control Firmness. Range 0 to 100	@01:PGAINF #01:PGAINF=80
VGAINF	RW	Velocity Control Firmness. Range 0 to 100	@01:VGAINF #01:VGAINF=80
KFCF	RW	Kalman filter constant for current noise	@01:KFCF #01:KFCF=10
KFNF	RW	Kalman filter constant for velocity/encoder noise	@01:KFNF #01:KFNF=10

A.5. Standalone Program Commands

Command	RW*	Description	
GOSUB	W	Run subroutine Subroutine from 1 to 32	@01:GOSUB=1 #01:GOSUB=1
SAC	W	Standalone program control 1 – run all programs 2 – stop all programs 3 – pause all programs 4 – continue all programs 10 – store standalone programs and auto run to flash 40 – run program 0 41 – run program 1 42 – run program 2 43 – stop program 0 44 – stop program 1 45 – stop program 2 46 – pause program 0 47 – pause program 1 48 – pause program 2 49 – continue program 0 50 – continue program 1 51 – continue program 2	@01:SAC=1 #01:SAC=1
SASM[#]	R	Standalone program status of program 0, 1, or 2 0 – Idle 1 – running 2 – Paused 4 – Errored For program 0, use SASM0 command For program 1, use SASM1 command For program 2, use SASM2 command	@01:SASM[0]=1 #01:SASM[0]=1

A.6. Limit Commands

Command	RW	Description	Example
LHPOL	RW	Limit and Home input polarity and limit enable Bit 0 – Home Input Polarity Bit 1 – Limit Input Polarity Bit 2 – Enable/Disable Hard Limit	@01:LHPOL #01:LHPOL=3
LIMPRO	RW	Sets action to perform when soft or hard limit is triggered 0 – Disable Motor 1 – Immediate Stop Motor with Servo On	@01:LIMPRO #01:LIMPRO=0
SLIMNEG	RW	Negative Soft Limit Value	@01:SLIMNEG #01:SLIMNEG=0
SLIMON	RW	Enable Soft Limit	@01:SLIMON #01:SLIMON=0
SLIMPOS	RW	Positive Soft Limit Value	@01:SLIMPOS #01:SLIMPOS=10000

A.7. IO Commands

Command	RW*	Description	Example
DIN	R	Digital Inputs. TITAN has 8 digital inputs. Depending on whether TITAN is set in controller or pulse mode, some digital inputs have special functions. DI1 – Pulse (Pulse Mode) DI2 – Dir (Pulse Mode) DI3 – Enable (Pulse Mode) DI4 – Clear Fault (Pulse Mode) DI5 – Reset Pos (Pulse Mode)/ Emergency Input (both) DI6 – Plus Limit (Controller Mode) DI7 – Home (Controller Mode) DI8 – Minus Limit (Controller Mode) **Return value in Hex 0x0 format**	@01:DIN #01:DIN=0x0
DOA	RW	Digital Output DO1 - bit 0	@01:DOA=1 #01:DOA=1 @01:DOA #01:DOA=1
DOB	RW	Digital Output DO2 - bit 1	@01:DOB=1 #01:DOB=1 @01:DOB #01:DOB=1
DOC	RW	Digital Output DO3 - bit 2	@01:DOC=1 #01:DOC=1 @01:DOC #01:DOC=1
DOUT	RW	Digital Outputs. TITAN comes with 3 digital outputs. Depending on whether TITAN is set in controller or pulse mode, some digital outputs have special functions. DO1 – Alarm (Pulse Mode) DO2 – Available for use DO3 – In-Pos (Pulse Mode) **Return value in Hex 0x0 format**	@01:DOUT #01:DOUT=0x0 @01:DOUT=0x3 #01:DOUT=0x3
AIN	R	Analog input status TITAN has one 12-bit analog input. Return value range [0-4095].	@01:AIN #01:AIN=4

A.8. Variable Commands

Command	RW	Description	Example
V[n]	RW	Set Variable Number to Read or Write	@01:V[0] #01:V[0]=0 @01:V[0]=12345 #01:V[0]=12345

Notes:

V[0] is equivalent to variable V1,
V[1] is equivalent to variable V2, etc.

A.9. Fault Monitoring Commands

Command	RW	Description	Example
CEMS	RW	Current Error Duration in Milliseconds	@01:CEMS #01:CEMS=1000
CEVAL	RW	Current Error Value in milliamps	@01:CEVAL #01:CEVAL=1000
EANG	R	Bias Angle Value	@01:EANG #01:EANG=1.25
ENABIAS	RW	Enable Bias Angle Monitoring (Use EANG to read the angle)	@01:ENABIAS #01:ENABIAS=1
ENAFC	RW	Enable Position and Current Fault Monitoring Bit 0 – Enable Position Error Monitoring Bit 1 – Skip In Position Check Bit 2 – Enable Stall Current Monitoring Bit 3 – Encoder Break Check Bit 4 – Check Encoder Count per Index	@01:ENAFC #01:ENAFC=5
ESTOP	RW	Digital Input 5 configured as an emergency input. When this input is triggered, the motor is disabled and fault flag is set. Value 0 – Disable emergency input Value 1 – Enable emergency active high Value 2 – Enable emergency active low	@01:ESTOP #01:ESTOP=0
OCUR	RW	Peak Motor Current Monitoring Value. The default value of 1**8A** is always set at power-up. Peak current monitoring is always enabled for the protection of the motor and driver. **OCUR** value can be changed to a lower current threshold value. **OCCUR** cannot be raised above 1**8A**.	@01:OCUR #01:OCUR=18
OVOL	RW	Peak Power Supply Voltage Monitoring Value. The default value of **65V** is always set at power-up. Peak voltage monitoring is always enabled for the protection of the motor and driver. **OVOL** value can be changed to set to a lower voltage threshold value. **OVOL** cannot be raised above **65V**.	@01:OVOL #01:OVOL=65
PEMS	RW	Position Error duration in milliseconds.	@01:PEMS #01:PEMS=1000
PEVAL	RW	Position Error Value in encoder counts.	@01:PEVAL #01:PEVAL=1000
OHEA	RW	Overheat Temperature Monitoring Value. The default value of **80 deg C** is always set at power-up. Temperature monitoring is always enabled for the protection of the motor and driver. **UHEA** can be lowered but not be raised higher than 80.	@01:OHEA #01:OHEA=80
UVOL	RW	Under Power Supply Voltage Monitoring Value. The default value of 5**V** is always	@01:UVOL #01:UVOL=5

		set at power-up. Under-voltage monitoring is always enabled for the protection of the motor and driver. **UVOL** value can be changed to set to a higher voltage threshold value. **UVOL** cannot be lower than 5**V** or raised above **65V**.	

A.10. Miscellaneous Commands

Command	RW	Description	Example
FIRMVS	R	Returns firmware version.	@01:FIRMVS #01:FIRMVS=571
HARDVS	R	Returns hardware version.	@01:HARDVS #01:HARDVS=101
PRODID	R	Returns Product ID	@01:PRODID #01:PRODID=301
MCUR	RW	Returns or sets the max current of the motor. Max current is used only during the acceleration and deceleration of the motion.	@01:MCUR #01:MCUR=3.0 @01:MCUR=3.5 #01:MCUR=3.5
MOTNAME	RW	Returns or sets motor product name. No space or tab character allowed. The underscore character will be treated as space character.	@01:MOTNAME #01:MOTNAME=EE17-3
NETID	RW	Returns and sets RS485 Network ID. The range is 1 to 99. Once the Network ID is set, it must be stored to flash, and the controller should reboot for the new ID to take effect.	@01:NETID #01:NETID=1
PWRC	R	Power source current usage.	@01:PWRC #01:PWRC=0.149019
PWRV	R	Power source voltage.	@01:PWRV #01:PWRV=24.1112
RCUR	RW	Returns or sets the rated current of the motor.	@01:RCUR #01:RCUR=2.5 @01:RCUR=2.8 #01:RCUR=2.8
RESET	W	Power cycles the controller. **No reply is returned.** On the USB port, when the power boots up, it will announce the product name. RS485 does not send out any announcement.	@01:RESET TITAN-SVX-ETH
STORE	W	Stores all parameters to flash memory. Once the store command is issued, wait at least 500msec before communication started.	@01:STORE #01:STORE=1
TEMP	R	Returns the current temperature of the unit.	@01:TEMP #01:TEMP=33.3302
SYSN	R	Returned Product Serial ID. This is read only.	@01:SYSN #01:SYSN=ASM305201008035901
SYSTIME	RW	Returns the number of seconds since the last power-up.	@01:SYSTIME #01:SYSTIME=166
RUNTIME	RW	Returns the number of seconds the motor is enabled and moving (velocity non-zero). Value is reset to zero at power-up.	@01:RUNTIME #01:RUNTIME=0

A.11. Probe Commands

Command	RW	Description	Example
PROBEA PROBEB PROBEC	RW	Selects data type for Probe A/B/C Mode: 0 – none 1 – Actual Position 2 – Actual Velocity 3 – Target Position 4 – Target Velocity 5 – Position Error 6 – Velocity Error 7 – Actual Current 8 – Target Current	@01:PROBEA=1 #01:PROBEA=1 @01:PROBEB #01:PROBEB=3
PROBET	RW	Probe Delta Time – Sets the time between the probing. Value is in msec.	@01:PROBET=1 #01:PROBEA=1 @01:PROBET #01:PROBEB=1
PROBECMD	W	Sets the probe command: 1 – start collecting probe data. 2 – stop collecting probe data.	@01:PROBECMD=1 #01:PROBECMD=1
PROBEST	R	Probe Status. 0 – IDLE 1 – Collecting 2 – Done	@01:PROBEST #01:PROBEST=2
BUFA BUFB BUFC	R	Reads the probed values. 250 values total, starting from index 0 to 254.	@01:BUFA[0] #01:BUFA[0]=125 @01:BUFB[0] #01:BUFB[0]=4321

A.12 Using TITAN-ASCII Command

All TITAN-ASCII protocol commands start with the character **'@'** and two network ID characters plus a separation character **':'** with CR and LF characters appended at the end.

All TITAN-ASCII protocol replies start with the character **'#'** and two network ID characters plus a separation character **':'** with CR and LF characters appended at the end.

All example commands shown are in Communication 0.

A.12.1. Position, Velocity, Position Error

To read and write the current encoder position, use the **EX** command.

To read motor encoder position
Command: **@01:EX**
Reply: **#01:EX=12345**
To set motor encoder position
Command: **@01:EX=54321**
Reply: **#01:EX=54321**
Note:

- Encoder value is in encoder counts.
- **EX** is read/write command.

To read the current target position, use the **POSD** command.

To read motor target position
Command: **@01:POSD**
Reply: **#01:POSD=12345**
Note:

- Target position value is in encoder counts.
- When the servo is off, **POSD** does not have any meaning since the target position is undetermined.
- **POSD** is a read-only command.

To read the current encoder position error, use the **PERR** command.

To read motor encoder error
Command: **@01:PERR**
Reply: **#01:PERR=0**
Note:

- Position error is the difference between the target or profile position **POSD** and the actual position **EX**.
- **PERR** is a read-only command.

A.12.2. In-Pos

To read and write the in-position range value, the **INPOSVAL** command.

To read in-position value:

Command: **@01:INPOSVAL**
Reply: **#01:INPOSVAL=100**

To set in-position value:

Command: **@01:INPOSVAL=200**
Reply: **#01:INPOSVAL=200**

Note:

- The in-position value is in encoder counts and is equal to the target minus the actual encoder position.
- The in-position value is used in the In-Pos bit on the **MST** motor status value.
- INPOSVAL is the read/write command.

A.12.3. Velocity

To read the current encoder velocity, use the **VX** command.

To read the motor encoder position:

Command: **@01:VX**
Reply: **#01:VX=100**

Note:

- The velocity value is read-only.
- The velocity value will be RPM (**rev/min**) or **mm/sec,** depending on the motor setup.
- **VX** command is a read-only command.

A.12.4. Motor Current

To read the motor current, use the **CURQA** and **CURDA** commands.

To read the motor current values:

Command: **@01:CURQA;CURDA**
Reply: **#01:CURQA=-0.009;CURDA=0.002**

Note:

- Values of the motor current are in Amperes.
- For the current reading of atypical normal motion, use the **CURQA** value to read the current that is flowing through the motor to control the motion. CURDA is a useful current value to read when in high-speed motion.
- **CURQA** and **CURDA** are read-only commands.

A.12.5. Motor Status

To read the status, use the command **MST**. The reply to this command will be a hex number that represents the motor status, with each bit representing the various states of the motion.

Bit	Name
0	Enabled
1	In-Position
2	Moving
3	In Fault
4	Homed

MST Value	Enabled	In Pos	Moving	Fault	Homed	Description
0x0	0	0	0	0	0	The motor power is off and disabled. To enable the servo, use the **SVON** command. Before any motion, the servo must be on.
0x1	X	0	0	0	0	The motor is enabled and in closed loop servo, but not moving and not in position. In-Pos range can be increased using the **INPOSVAL** command.
0x3	X	X	0	0	0	The motor is enabled and in closed loop servo, and the current position is within in-pos value of the target. To turn off the servo, use the **SVOFF** command
0x4	0	0	X	0	0	The motor is disabled but moving. This situation should not arise unless the motor is disabled but it is still moving.
0x5	X	0	X	0	0	The motor is enabled and moving. To stop the motion, use the **STOPX** command
0x8	0	0	0	X	0	The motor has a fault status. The fault must be cleared before any motion can be done. To clear the fault, use the **ECLEARX** command.
0x11	1	0	0	0	1	The motor is enabled and the homing was completed successfully.

To read the motor status:

Command: **@01:MST**
Reply: **#01:MST=0x3**

Note:

- **MST** is a read-only command.

A.12.6. Motor Fault

To check the motor's status, use the **MST** command. Bit 4 indicates whether the motor has a fault status. If the motor is in a fault state, the **FLT** command can be used to determine what type of fault caused the fault state to be triggered.

To read the fault status of the motor, use the command **FLT**. The reply to this command will be a hex number that represents the fault state of the motor.

When a fault occurs, the motor is disabled immediately. The fault must be cleared before the servo can be turned on or before any motion can be performed.
The fault state of the motor can be cleared using the **ECLEARX** command.

To read the motor's state:

Command: **@01:MST**
Reply: **#01:MST=0x8**

To read the motor's fault state:

Command: **@01:FLT**
Reply: **#01:FLT=0x3**

Bit	Name	Description
0	Negative Limit Error	Error applies to both soft limit and hard limit triggering. Soft limit monitoring and hard limit monitoring can be enabled and disabled using the **SLIMON** and **LHPOL** commands.
1	Positive Limit Error	Error applies to both soft limit and hard limit triggering. Soft limit monitoring and hard limit monitoring can be enabled and disabled using the **SLIMON** and **LHPOL** commands.
2	Position Error	When the motor is enabled and the difference between the target and actual position is greater than the position error value for a set duration of time, this fault is triggered. Use the **ENAFC** command to enable and disable Position Error Check. Use the **PEMS** and **PEVAL** commands to set the duration and value of the position error checking.
3	Current Error	When the motor is enabled and the current supplied to the motor is greater than the set current error value for a set duration of time, this fault is triggered. Use the **ENAFC** command to enable and disable current error check. Use the **CEMS** and **CEVAL** commands to set the duration and value of the current error checking.
4	Hall Sensor Error	Hall sensor error detection is done only for the motor that has been configured to use the hall sensor. When the hall sensor is used, a minimum of one or two of UVW should be on. When there is no signal or all three hall sensor signals are on, this fault is triggered. Hall sensor error checking is enabled all the time when the motor is configured to use the hall sensor.
5	Peak Motor Current	Peak current has been triggered. The motor is disabled

	Error	when this error occurs. Peak current value can be changed usingthe**OCUR** command.
6	Peak Supply Voltage Error	Peak power supply voltage has been triggered. Typically, this happens when the back EMF of the motor raises the power supply voltage. The motor is disabled when this error occurs. The Peak Voltage Value can be changed usingthe**OVOL** command.
7	Under Power Supply Voltage Error	Under power voltage has been triggered. The motor is disabled when this error occurs. Theunderpowervoltagevalue can be changed usingthe**UVOL** command.
8	Overheat Error	The approximate FET temperature is measured, and the maximum threshold value erroristriggered if the temperature goes above **80 deg C**. Usethe**TEMP** command to read the current temperature. Usethe**UHEA**commandto read the threshold temperature value.
9	Motor Connection Error	The motor connection monitoring is done when the motor is enabled. Actual and target current and voltage values are monitored to determine if the motor connection is not present.
10	Emergency Switch On	Digital input 5 can be configured as emergency input. When this input trigger is detected, the motor is disabled and fault flag is set.

Table A.15

Note:

- **FLT** is a read-only command.
- Use the **ECLEARX** command to clear the fault.
- Use the **MST** command to determine whether the motor's status is in fault.

A.12.7. Enable Servo / Open Loop Position Hold

SVON and **SVOFF** are the two commands used to turn the motor servo on and off.

To turn the servo on:

Command: **@01:SVON**
Reply: **#01:SVON=1**

To turn the servo off:

Command: **@01:SVOFF**
Reply: **#01:SVOFF=1**

Note:

- When the motor servo is on, the motor is in the enable state or the servo is in the on state, in-pos state can be monitored using the **MST** command.
- **SVON and SVOFF** are write-only commands.

OLPHOLD is also known as the Open Loop Position Hold command. The **OLPHOLD** command is valid **ONLY** for stepper motors. When **OLPHOLD** is zero, full time closed loop is performed at Enabled and Not Moving state. When **OLPHOLD** is a non-zero value between 1 and 100, current is applied to the stepper motor coils at 1% to 100% of the maximum current.

NOTE:
When **OLPHOLD** is used, constant current is applied to the motor which will cause the motor to heat up. Use **OLPHOLD** for a short duration of time.

To disable OLPHOLD and perform full time servo:

Command: **@01:OLPHOLD=0**
Reply: **#01:OLPHOLD=0**

To enable OLPHOLD at 50% of the rated current:

Command: **@01:OLPHOLD=50**
Reply: **#01:OLPHOLD=50**

A.12.8. Jogging

JOGV, JOGXP and **JOGXN** are the jogging commands used to jog the motor in a positive or negative direction.

For speed, acceleration, and deceleration values, **HSPD** command is used for target speed, **ACC** command for acceleration, and **DEC** command for deceleration.

By default, after each **ACC** command, the **DEC** is set to the same value as **ACC** value. To use the deceleration value, issue the **DEC** command after the **ACC** command.

For **JOGV** command, the target speed is set with the **JOGV** command.
.
Before any motion can be initiated, the motor must be in a servo on state. Use **SVON** to enable and turn on the servo of the motor.

The **STOPX** command is used to stop the motor in motion.

To turn the servo on:
Command: **@01:SVON**
Reply: **#01:SVON=1**

To set the target speed:
Command: **@01:HSPD=500**
Reply: **#01:HSPD=500**

To set the acceleration:
Command: **@01:ACC=2000**
Reply: **#01:ACC=2000**

To set the acceleration:
Command: **@01:DEC=3000**
Reply: **#01:DEC=3000**

To jog the motor in positive direction:
Command: **@01:JOGXP**
Reply: **#01:JOGXP=1**

To stop the jogging motor:
Command: **@01:STOPX**
Reply: **#01:STOPX=1**

To jog the motor in negative direction:
Command: **@01:JOGX=N**
Reply: **#01:JOGXN=1**

To check the motor status:
Command: **@01:MST**
Reply: **#01:MST=0x5**

A.12.9. Homing

HMODE and **HOMEX** are the commands used to perform the home search of the motor.

Two other commands used in homing are the target speed **HSPD** and the acceleration **ACC** commands.

Before any motion can be initiated, the motor must be in a servo-on state. Use **SVON** command to enable and turn on the servo of the motor.

The **STOPX** command is used to stop the motor in motion.

To set the homing mode to search in minus direction for the encoder index:
Command: **@01:HMODE=9**
Reply: **#01:HMODE=9**

To turn the servo on:
Command: **@01:SVON**
Reply: **#01:SVON=1**

To set the target speed:
Command: **@01:HSPD=500**
Reply: **#01:HSPD=500**

To set the acceleration:
Command: **@01:ACC=2000**
Reply: **#01:ACC=2000**

To home the motor in using the set homing mode:
Command: **@01:HOMEX**
Reply: **#01:HOMEX=1**

To stop the motor:
Command: **@01:STOPX**
Reply: **#01:STOPX=1**

To check the motor status:
Command: **@01:MST**
Reply: **#01:MST=0x5**

To check the motor position value:
Command: **@01:EX**
Reply: **#01:EX=0**

A.12.10. Target Move

The **X** command is used to perform a target position move of the motor.

For speed, acceleration, and deceleration values, **HSPD** command is used for target speed, **ACC** command for acceleration, and **DEC** command for deceleration.

By default, after every **ACC** command, the deceleration value is set to the same value as the acceleration value. To use the deceleration value, issue the **DEC** command after the **ACC** command.

Before any motion can be initiated, the motor must be in a servo-on state. Use **SVON** to enable and turn on the servo of the motor.

The **STOPX** command is used to stop the motor in motion.

To turn the servo on:

Command: **@01:SVON**
Reply: **#01:SVON=1**

To Set the target speed:

Command: **@01:HSPD=500**
Reply: **#01:HSPD=500**

To set the acceleration, use the **ACC** command. The deceleration value is set to the same value as the acceleration value at every **ACC** command:

Command: **@01:ACC=2000**
Reply: **#01:ACC=2000**

To set different deceleration value different from the acceleration value, issue the **DEC** command after the ACC command:

Command: **@01:ACC=2000**
Reply: **#01:ACC=2000**

To move the motor to the target position 25000:

Command: **@01:X=25000**
Reply: **#01:X=1**

To stop the motor:

Command: **@01:STOPX**
Reply: **#01:STOPX=1**

To check the motor status:

Command: **@01:MST**
Reply: **#01:MST=0x5**

To check the motor position value:

Command: **@01:EX**
Reply: **#01:EX=25000**

A.12.11. Gains

TITAN uses simplified gain commands, also known as firmness value. Instead of using potentially undecipherable gain values, simplified firmness value ranges from 0 (soft) to 100 (hard). Note that 0 does not mean azero gain value but rather, the lowest value in the firmness range.

Three firmness commands are available related to closed loop motion control:

PGAINF – Position-related Firmness value
VGAINF – Velocity-related Firmness value
IGAINF – Integral-related Firmness value

Motor current closed loop control Firmness is also available.

CGAINF – Current related gain firmness value

To set the position control firmness:

Command: **@01:PGAINF=80**
Reply: **#01:PGAINF=80**

To set the current control firmness:

Command: **@01:ACC=2000**
Reply: **#01:ACC=2000**

Note:

- Firmness values can be set atanytime, even during motion.
- The firmness value takes effect at the command request.

A.12.12. Digital IO

TITAN has 8 digital inputs and 3 digital outputs. Digital inputs can be read by using the **DIN** command. Digital outputs can be read and set using the **DOUT** command.

To check the digital input values:

Command: **@01:DIN**
Reply: **#01:DIN=7**

To check the digital output values:

Command: **@01:DOUT**
Reply: **#01:DOUT=0**

To set the digital output value to 3 (turn on first two bits):

Command: **@01:DOUT=3**
Reply: **#01:DOUT=3**

A.12.13. LED

TITAN has three internal LED outputs (Red, Green, and Blue) and one front-face blue LED.

By default, the front-face LED is turned on when TITAN is powered on. Command LED is used to check and set the front LED.

By default, the three internal LED will turn on in a different color depending on the motor type. The command RGB is used to check and set the three internal LEDs.

To check the RGB LED status:

Command: **@01:RGB**
Reply: **#01:RGB=4**

To set the turn on the green internal LED:

Command: **@01:RGB=2**
Reply: **#01:RGB=2**

To turn off the front-face blue LED:

Command: **@01:LED=0**
Reply: **#01:LED=0**

A.12.14. Variable

TITAN has 100 variables V1 to V100 (V[0] to V[99] in array format) that can be read and set. Variables can also be used in the standalone programs running on the TITAN as V1 to V100. Variables can be used to hold values, to perform math operations, and to control motion, such as when moving to a variable value.

Variable array command can be used to read and write the variables.

To read example**:**

Command: **@01:V[23]=123**
Reply: **#01:V[23]=123**

To write example:

Command: **@01:V[23]**
Reply: **#01:V[23]=123**

A.12.15. Program

TITAN supports three standalone programs that can run in multi-thread. Standalone programs need to be written and compiled and downloaded from the TITAN windows program. Standalone programs can be stored in the flash memory so that the program can be run after the power cycle.

The standalone control command **SAC** can be used to start, stop, pause, continue any one or all three standalone programs.

The **SASM** command is used to check the status of the program, whether it is running, paused, or in error.

Start running all three programs:
Command: **@01:SAC=1**
Reply: **#01:SAC=1**

Stop all three programs:
Command: **@01:SAC=2**
Reply: **#01:SAC=2**

Pause all three programs:
Command: **@01:SAC=3**
Reply: **#01:SAC=3**

Continue all three programs:
Command: **@01:SAC=4**
Reply: **#01:SAC=4**

Start running program 0:
Command: **@01:SAC=40**
Reply: **#01:SAC=40**

Stop running program 0:
Command: **@01:SAC=43**
Reply: **#01:SAC=43**

Note:
When using the program commands, **DO NOT USE**a multiple command format using the ";" character. Use single command format for standalone program control.

A.12.16. Miscellaneous Commands

Get network ID:
Command: **@01:NETID**
Reply: **#01:NETID=1**

Set network ID:
Command: **@01:NETID=2**
Reply: **#01:NETID=2**

Store parameters to flash including the network ID:
Command: **@01:STORE**
Reply: **#01:STORE=1**

Soft power cycle TITAN:
Command: **@01:RESET**
Reply: (no reply, TITAN will perform power cycle)

Get firmware version:
Command: **@01:FIRMVS**
Reply: **#01:FIRMVS=401**

Get power supply voltage:
Command: **@01:PWRV**
Reply: **#01:PWRV=24.5**

Get power supply current:
Command: **@01:PWRC**
Reply: **#01:PWRC=0.120**

Get the motor name that has been configured:
Command: **@01:MOTNAME**
Reply: **#01:MOTNAME=NMB23-2**

A.12.17. Multi-axis Synchronous Start Move

The **CM** command is used to perform a synchronous multi-axis target position move.

Total of 6 axes can be moved synchronously using the CM command in a broadcast using address 00.

Each target position is represented by 8 character hex value string. For example target position of 1000 is 000003E8. Target position of -5000 is FFFEC78.

For CM command use the broadcast command to issue the commands to all 6 axis synchronously.

Before any motion can be initiated, the motor must be in a servo-on state. Use **SVON** to enable and turn on the servo of the motor.

The **STOPX** command is used to stop the motor in motion.

To turn the servo on for all axes:
Command: **@00:SVON**

To Set the target speed to all axes:
Command: **@00:HSPD=500**

To set the acceleration to all axes:
Command: **@00:ACC=2000**

To move the motor 1 to 1000, motor 2 to -5000, and motor 3 to 2000:
Command: **@00:CM=000003E8FFFFEC78000007D0**

To stop the all axes:
Command: **@00:STOPX**

A.12.18. Motion Probe

TITAN controller comes with built in motion probing for accurate capture of the motion performance.

Total of 3 sets of values can be captured with each set having 250 captured values.

To setup the probe capture, use the **PROBEA**, **PROBEB**, and **PROBEC** commands to setup the capture types. Following are the possible values for the setup commands:

0 – none
1 – Actual Position
2 – Actual Velocity
3 – Target Position
4 – Target Velocity
5 – Position Error
6 – Velocity Error
7 – Actual Current
8 – Target Current

Probe capture interval can be set from 1 msec to 50 msec using the **PROBET** command. If 1 msec is used, total of 3 sets of 250msec values are captured at 1 msec interval. If 50 msec is used, total of 3 sets of 5 x 250msec value are captured at 50 msec interval.

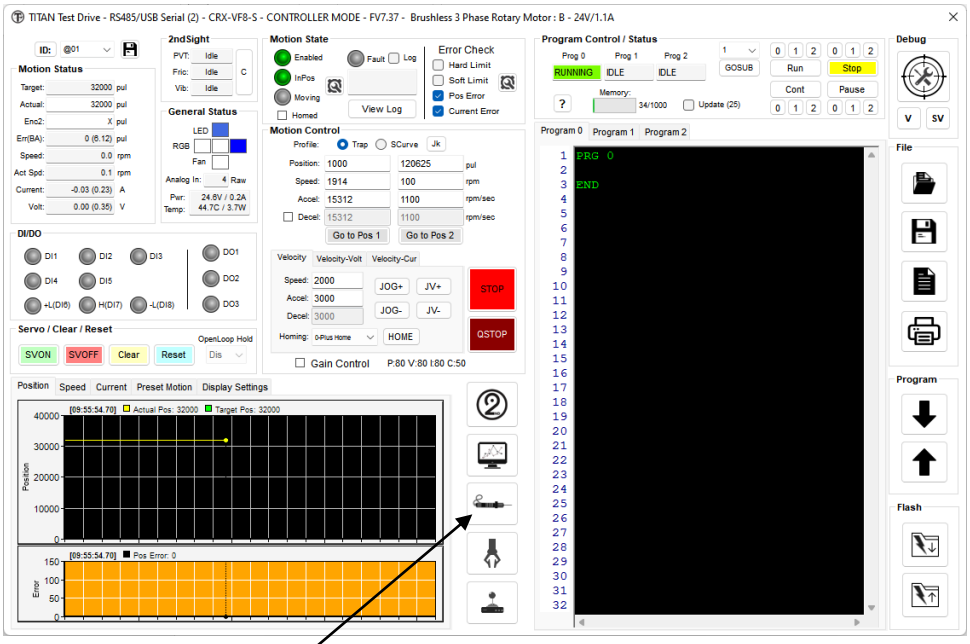
Once the probe capture is done, the probe status can be read using the **PROBEST** command. Following are possible values for probe status:

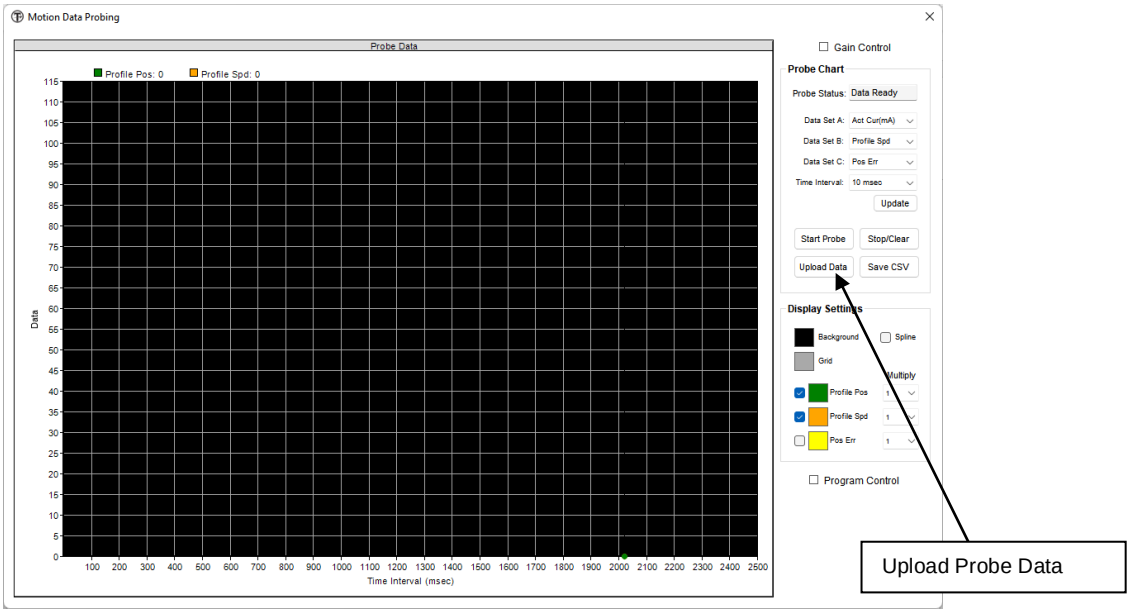
0 – IDLE
1 – Collecting
2 – Done

Captured values can be read using the **BUFA**, **BUFB**, and **BUFC** commands. These are array values. Range of array values are 0 to 249 for total of 250 values per set. To read, use the format:

BUFA[array number]
BUFB[array number]
BUFC[array number]

Note that the probe values can be read and displayed from the TITAN Software by going to the probe screen and uploading the values.





Following are examples of probe commands.

To set the probe A to capture the position error:

Command: **@01:PROBEA=5**
Reply: **#01:PROBEA=5**

To set the 5 msec probe capture interval time:

Command: **@01:PROBET=5**
Command: **#01:PROBET=5**

To read the probe status:

Command: **@01:PROBEST**
Reply: **#01:PROBEST=2**

To read the three probed values from buffer A:

Command: **@01:BUFA[0];BUFA[1];BUFA[2]**
Reply: **#01:BUFA[1]=12;BUFA[1]=12;BUFA[2]=15**

Contact Information

Arcus Servo Motion, Inc.

3159 Independence Drive
Livermore, CA 94551
925-373-8800

www.arcusservo.com